MOR Practical Session

Report – Session 2

Solving the Unsteady Heat Equation
with Proper Generalized Decomposition (A Priori)

ENS Paris-Saclay – $(MS)^2SC$ Master

Eren Alagoz (Instructor)

March 1, 2026

Contents

1 Problem Statement **2**

2 Reference Solution: FEM with Backward Euler **2**
2.1 Spatial Discretization . . . 2
2.2 Integration Operators . . . 2
2.3 Time Integration: Backward Euler . . . 2
2.4 Boundary Conditions by Substitution . . . 3

3 Change of Variable **3**
3.1 Motivation . . . 3
3.2 Splitting the Solution . . . 3
3.3 Modified Equation for W . . . 3
3.4 Validation . . . 4

4 PGD Resolution **4**
4.1 Principle of the Separated Representation . . . 4
4.2 Weak Form in Space-Time . . . 4
4.3 Fixed-Point Algorithm . . . 4
4.3.1 Spatial Sub-Problem (λ fixed) . . . 4
4.3.2 Temporal Sub-Problem (Λ fixed) . . . 5
4.3.3 Stagnation Criterion . . . 5
4.4 Greedy Algorithm . . . 5
4.5 Error Computation . . . 5

5 Results **6**
5.1 Reference Solution . . . 6
5.2 Change of Variable Validation . . . 6
5.3 PGD Convergence . . . 6
5.4 PGD vs. Reference Solution . . . 7

6 Complexity Analysis **7**

7 Conclusion **7**

1 Problem Statement

We consider the 1D unsteady heat equation on a bar $\Omega = [0, L]$ over time interval $I = [0, T]$:

$$\rho c \frac{\partial u}{\partial t} - k \frac{\partial^2 u}{\partial x^2} = f(x,t) \quad \text{in } \Omega \times I \tag{1}$$

with oscillating Dirichlet boundary conditions $u(0,t) = \sin(2\pi t/T)$ and $u(L,t) = -\sin(4\pi t/T)$, and initial condition $u(x,0) = 0$.

The physical parameters are $\rho = c = k = 1$, $L = 1$, $T = 2$, with $n_X = 300$ spatial nodes and $n_T = 100$ time steps. The volume loading f is chosen randomly.

The objective is to solve this equation using PGD and compare the result with a classical Finite Element reference solution.

2 Reference Solution: FEM with Backward Euler

2.1 Spatial Discretization

We approximate $u(x,t) \approx \sum_j U_j(t)\,\varphi_j(x)$ using P1 hat functions φ_j. The standard Galerkin weak form yields the semi-discrete system:

$$M\dot{U}(t) + KU(t) = F(t) \tag{2}$$

where $M_{ij} = \int_\Omega \rho c\, \varphi_i \varphi_j \, dx$ is the consistent mass matrix, $K_{ij} = \int_\Omega k\, \varphi_i' \varphi_j' \, dx$ is the stiffness matrix, and $F_i = \int_\Omega f\, \varphi_i \, dx$ is the load vector.

2.2 Integration Operators

For computing L^2 norms and inner products in space and time, we assemble consistent integration matrices by looping over elements. The local element mass matrix is $\frac{h}{6}\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, assembled globally:

```
Ix = zeros(n_X); It = zeros(n_T);
Ie_x = dx * [1/3, 1/6; 1/6, 1/3];
Ie_t = dt * [1/3, 1/6; 1/6, 1/3];
for i = 1:(n_X - 1)
    Ix(i:i+1, i:i+1) = Ix(i:i+1, i:i+1) + Ie_x;
end
for i = 1:(n_T - 1)
    It(i:i+1, i:i+1) = It(i:i+1, i:i+1) + Ie_t;
end
```

Each element contributes its local 2×2 mass matrix to the global matrix at the corresponding DOFs. The overlapping assembly naturally produces the tridiagonal structure.

2.3 Time Integration: Backward Euler

The backward Euler scheme approximates $\dot{U}(t_i) \approx (U_i - U_{i-1})/\Delta t$:

$$\underbrace{\left(\frac{M}{\Delta t} + K\right)}_{H} U_i = F(t_i) + \frac{M}{\Delta t} U_{i-1} \tag{3}$$

2.4 Boundary Conditions by Substitution

The non-zero Dirichlet boundary conditions are enforced using the substitution method. The degrees of freedom are split into blocked DOFs ddld $= \{1, n_X\}$ and free DOFs ddlu. At each time step, the system is solved only on free DOFs:

```
for i = 2:n_T
    G = F(:,i) + (M/dt)*Uex(:,i-1);
    Uex(ddlu,i) = H(ddlu,ddlu) \ (G(ddlu) - H(ddlu,ddld)*Uex(ddld,i));
end
```

The term $H(\text{ddlu}, \text{ddld}) \cdot U_{\text{ex}}(\text{ddld}, i)$ transfers the influence of the known boundary values to the right-hand side.

3 Change of Variable

3.1 Motivation

PGD seeks $W(x,t) = \sum_i \Lambda_i(x) \cdot \lambda_i(t)$ where each spatial mode Λ_i must vanish at the boundaries. Since the actual solution U has non-zero boundary conditions, we cannot apply PGD directly to U.

3.2 Splitting the Solution

We decompose $U = U_{CL} + W$ where U_{CL} is a "dynamically admissible" field that satisfies the boundary and initial conditions, and W is the remainder with homogeneous BCs ($W(0,t) = W(L,t) = 0$).

The simplest choice is a linear interpolation:

$$U_{CL}(x,t) = \left(1 - \frac{x}{L}\right) \sin\left(\frac{2\pi t}{T}\right) + \frac{x}{L}\left(-\sin\left(\frac{4\pi t}{T}\right)\right) \tag{4}$$

In MATLAB, this is computed as an outer product:

```
u_cl = (1 - lx'/L) * Ud0 + (lx'/L) * UdL;
```

3.3 Modified Equation for W

Substituting $U = U_{CL} + W$ into (2):

$$M\dot{W} + KW = \underbrace{F - M\dot{U}_{CL} - KU_{CL}}_{Q_0} \tag{5}$$

The time derivative $\dot{U}_{CL}$ is computed using a finite difference operator acting on the right:

$$\mathbb{D}_t = \frac{1}{\Delta t}\begin{pmatrix} 1 & -1 & & \\ & 1 & -1 & \\ & & \ddots & \ddots \end{pmatrix} \tag{6}$$

so that `u_cl_dot = u_cl * Dt` computes the time derivative for each spatial node simultaneously. The modified right-hand side is then:

```
Dt = (1/dt) * (eye(n_T) + diag(-ones(n_T-1,1), 1));
u_cl_dot = u_cl * Dt;
G0 = F - M*u_cl_dot - K*u_cl;
```

3.4 Validation

We solve the modified equation (5) for W using the same backward Euler scheme, then recover $U_1 = W + U_{CL}$. The difference $\|U_1 - U_{\text{ex}}\|$ is of order 10^{-12}, confirming both approaches solve the same equation.

4 PGD Resolution

4.1 Principle of the Separated Representation

Instead of time-stepping, PGD seeks W as a sum of separated space-time modes:

$$W(x,t) = \sum_{i=1}^{m} \Lambda_i(x) \cdot \lambda_i(t) \tag{7}$$

Each mode (Λ_i, λ_i) is computed greedily, one at a time, using a fixed-point alternation between a spatial FE problem and a temporal ODE.

4.2 Weak Form in Space-Time

The PGD approximation is inserted into the space-time weak form of (5). Testing with $(\Lambda\lambda^* + \Lambda^*\lambda)$:

$$\int_0^T (\Lambda\lambda^* + \Lambda^*\lambda)^\top \left[M\Lambda\dot{\lambda} + K\Lambda\lambda - Q(t) \right] dt = 0 \tag{8}$$

where $Q = Q_0 - M\dot{W}_{\text{prev}} - KW_{\text{prev}}$ is the current residual.

4.3 Fixed-Point Algorithm

Since the problem is nonlinear in the pair (Λ, λ), we alternate:

4.3.1 Spatial Sub-Problem (λ fixed)

Setting $\lambda^* = 0$ in (8) and removing the arbitrary Λ^* yields:

$$\underbrace{\left[\left(\int_0^T \lambda\dot{\lambda}\, dt \right) M + \left(\int_0^T \lambda^2\, dt \right) K \right]}_{\mathbb{J}} \Lambda = \underbrace{\int_0^T Q(t)\lambda\, dt}_{q} \tag{9}$$

The time integrals are scalars computed from the known λ, so $\mathbb{J}$ is a linear combination of M and K — a standard $N_x \times N_x$ FE system solved only on free DOFs. After solving, Λ is normalized: $\Lambda \leftarrow \Lambda / \sqrt{\Lambda^\top I_x \Lambda}$.

In MATLAB, with `lambdatmp` as a $1 \times n_T$ row vector:

```
J = [lambdatmp*Dt*It*lambdatmp']*M + [lambdatmp*It*lambdatmp']*K;
q = lambdatmp * It * Q';
LAMBDAtmp = zeros(n_X, 1);
LAMBDAtmp(ddlu) = J(ddlu,ddlu) \ q(ddlu)';
LAMBDAtmp = LAMBDAtmp / sqrt(LAMBDAtmp'*Ix*LAMBDAtmp);
```

4.3.2 Temporal Sub-Problem (Λ fixed)

Setting $\Lambda^* = 0$ and removing the arbitrary λ^*:

$$\underbrace{\Lambda^\top M \Lambda}_{\alpha} \dot{\lambda} + \underbrace{\Lambda^\top K \Lambda}_{\beta} \lambda = \underbrace{\Lambda^\top Q(t)}_{\gamma(t)} \tag{10}$$

This is a scalar ODE solved by backward Euler at negligible cost:

$$\lambda_i = \frac{\gamma(t_i)\Delta t + \alpha\, \lambda_{i-1}}{\alpha + \beta\, \Delta t} \tag{11}$$

```
alpha = LAMBDAtmp'*M*LAMBDAtmp;
beta  = LAMBDAtmp'*K*LAMBDAtmp;
gamma = LAMBDAtmp'*Q;
lambdatmp = zeros(1, n_T);
for i = 2:n_T
    lambdatmp(i) = (gamma(i) + (alpha/dt)*lambdatmp(i-1)) / ((alpha/dt)
        + beta);
end
```

4.3.3 Stagnation Criterion

The alternation is stopped when λ no longer changes significantly:

$$s = \frac{\int_0^T (\lambda^{\text{new}} - \lambda^{\text{old}})^2\, dt}{\int_0^T (\lambda^{\text{old}})^2\, dt} = \frac{(\lambda^{\text{new}} - \lambda^{\text{old}})\, I_t\, (\lambda^{\text{new}} - \lambda^{\text{old}})^\top}{\lambda^{\text{old}}\, I_t\, (\lambda^{\text{old}})^\top} < 10^{-5} \tag{12}$$

4.4 Greedy Algorithm

The complete PGD algorithm is summarized below.

Algorithm 1 Greedy PGD (without update)

Initialize $W = 0$, $Q_0 = F - M\dot{U}_{CL} - KU_{CL}$
for mode $= 1, 2, \ldots, n_{\text{modes}}$ **do**
  Update residual: $Q = Q_0 - M\dot{W} - KW$
  Initialize λ (e.g., $\lambda = \texttt{lt}$)
  while $s > 10^{-5}$ and iterations < 10 **do**
    Solve FE problem (9) for Λ, normalize
    Solve ODE (10) for λ
    Compute stagnation s
  end while
  Store Λ and λ, update $W = \sum \Lambda_i \lambda_i$
  Compute error $\varepsilon = \|U_{\text{ex}} - U_{\text{PGD}}\|_{\mathcal{W}} / \|U_{\text{ex}}\|_{\mathcal{W}}$
end for

4.5 Error Computation

The relative error is measured in the space-time L^2 norm. The instructor's implementation integrates first in time for each spatial node, then in space:

```
error = zeros(n_X, 1);
norm_U_t = zeros(n_X, 1);
for i = 1:n_X
```

```
    error(i) = sqrt((Uex(i,:) - U_PGD(i,:)) * It * (Uex(i,:) - U_PGD(i
        ,:))');
    norm_U_t(i) = sqrt(Uex(i,:) * It * Uex(i,:)');
end
pgd_err = sqrt(error'*Ix*error) / sqrt(norm_U_t'*Ix*norm_U_t);
```

This computes $\varepsilon = \frac{\sqrt{\int_\Omega \left(\int_I (U_{\text{ex}} - U_{\text{PGD}})^2 \, dt\right) dx}}{\sqrt{\int_\Omega \left(\int_I U_{\text{ex}}^2 \, dt\right) dx}}$, which is the $L^2(\Omega \times I)$ norm.

5 Results

5.1 Reference Solution

The reference solution obtained by FEM + backward Euler is shown in Figure 1. The oscillating boundary conditions create a rich space-time pattern.

Surface plot of $U_{ex}(x,t)$

Figure 1: Reference solution U_{ex} computed by FEM + backward Euler.

5.2 Change of Variable Validation

The solution $U_1 = W + U_{CL}$ matches the reference to machine precision ($\sim 10^{-12}$), validating the change of variable approach.

5.3 PGD Convergence

The PGD error as a function of the number of modes is shown in Figure 2. The error decreases rapidly with the first few modes, then plateaus. This confirms that the solution manifold is low-dimensional, consistent with the theoretical expectations from the Kolmogorov n-width analysis.

Semilogy plot of ε vs. number of modes

Figure 2: PGD error convergence: relative error as a function of the number of modes.

5.4 PGD vs. Reference Solution

Figure 3 compares the reference and PGD solutions visually. After a sufficient number of modes, the PGD solution is visually indistinguishable from the reference.

Side-by-side: U_{ex} and U_{PGD}

Figure 3: Comparison between the reference (left) and PGD (right) solutions.

6 Complexity Analysis

The computational cost of each approach can be compared:

The classical FEM + Euler approach requires N_t linear solves of size N_x, each costing $\mathcal{O}(N_x^2)$ operations (for a banded system), giving a total cost of $\mathcal{O}(N_t \cdot N_x^2)$.

The PGD approach, for m modes with p fixed-point iterations each, requires $m \cdot p$ FE solves of size N_x (cost $\mathcal{O}(N_x^2)$ each) plus $m \cdot p$ ODE solves of size N_t (cost $\mathcal{O}(N_t)$ each). The total cost is $\mathcal{O}(m \cdot p \cdot (N_x^2 + N_t))$.

When $m \cdot p \ll N_t$, which is the typical regime for smooth problems, PGD is significantly more efficient.

7 Conclusion

This practical session demonstrated the PGD method as an a priori model order reduction technique for solving the 1D unsteady heat equation. The key steps were:

The change of variable $U = U_{CL} + W$ transforms the problem with non-homogeneous boundary conditions into one with zero BCs, enabling the separated representation.

The fixed-point alternation between a spatial FE problem and a temporal ODE efficiently computes each mode without assembling the full space-time system.

The greedy algorithm builds modes one by one, each correcting the residual left by the previous modes.

The PGD solution converges rapidly with a small number of modes ($m \ll N_t$), confirming that the solution manifold is intrinsically low-dimensional. This makes PGD particularly attractive for problems where the classical time-stepping approach would be prohibitively expensive, such as multi-parametric or real-time applications.